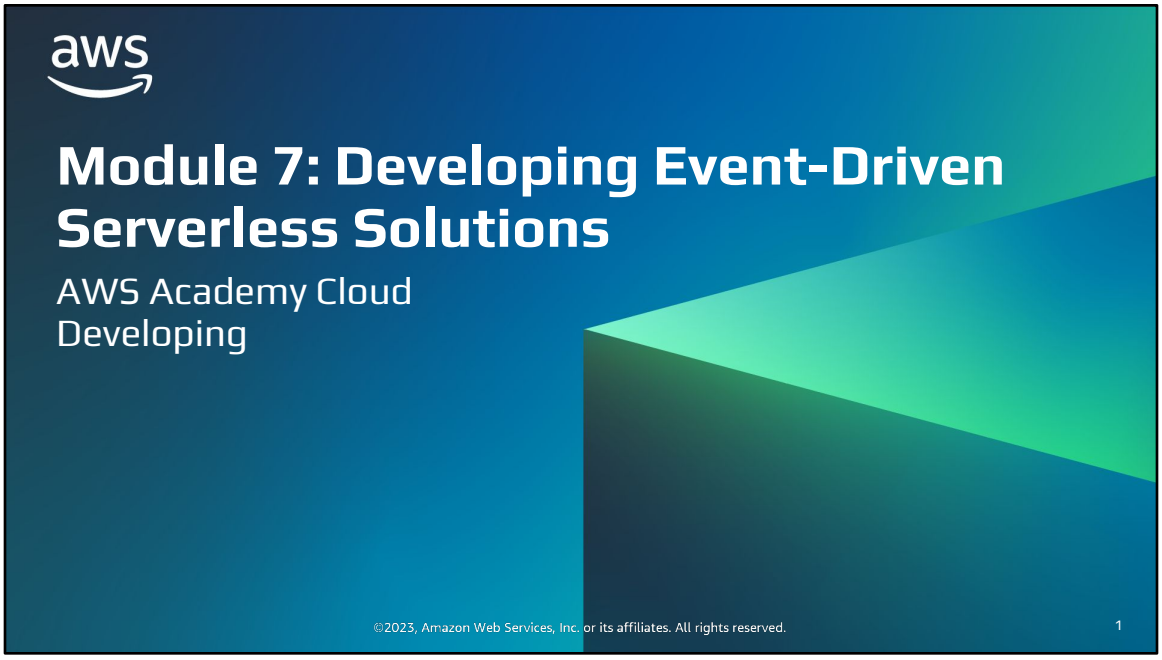
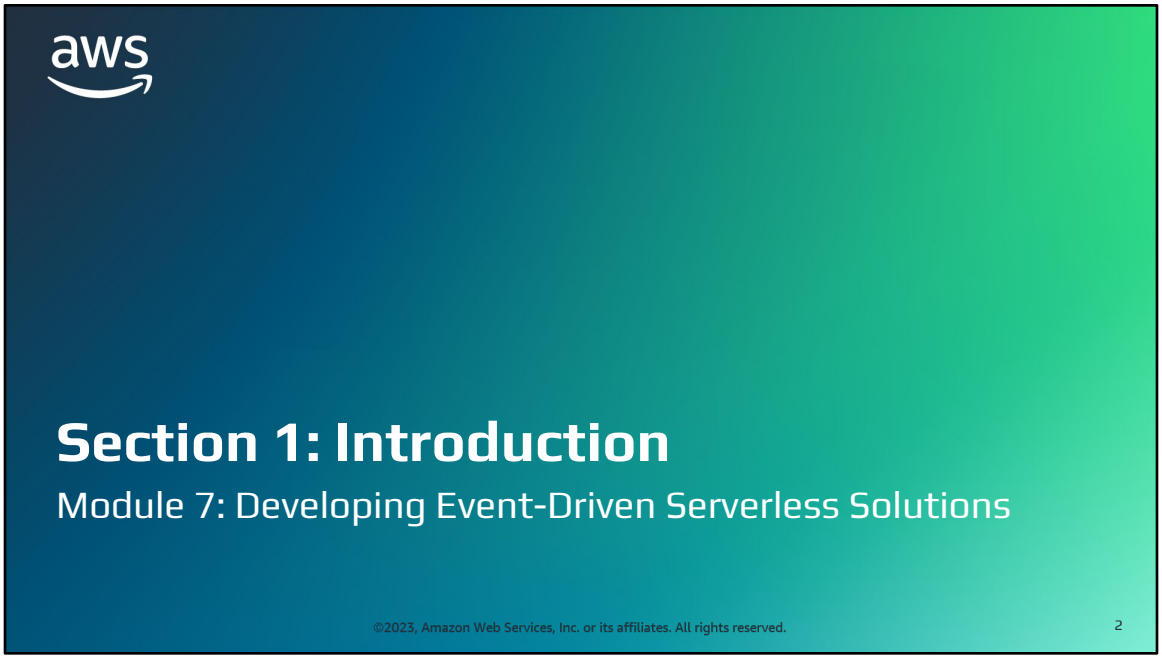
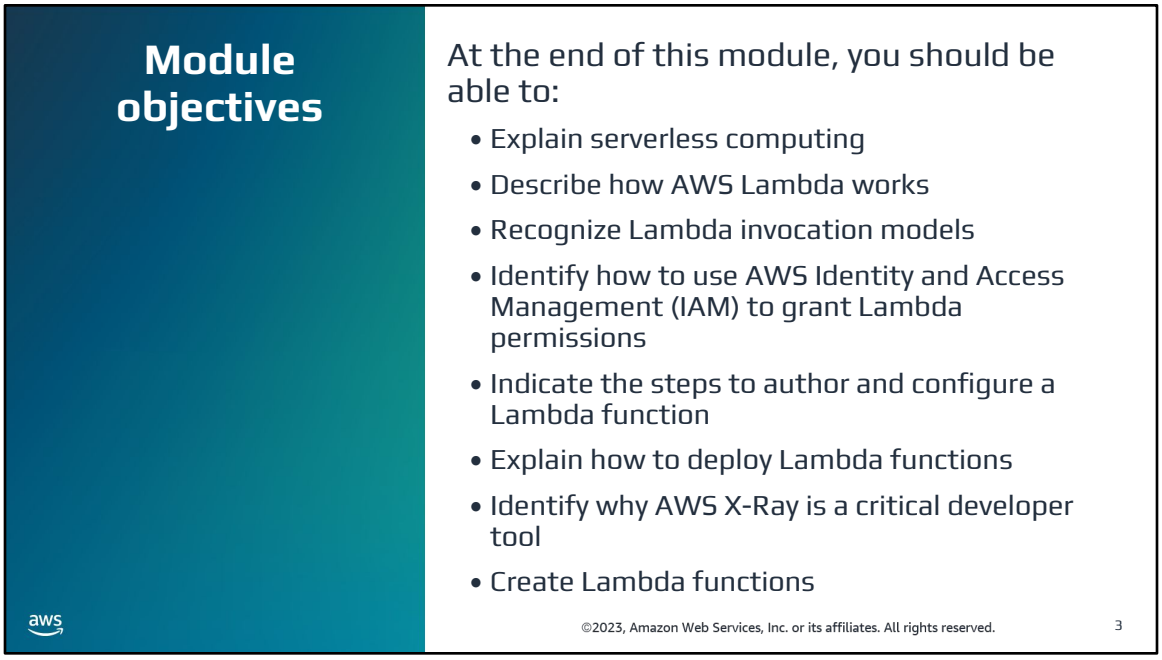


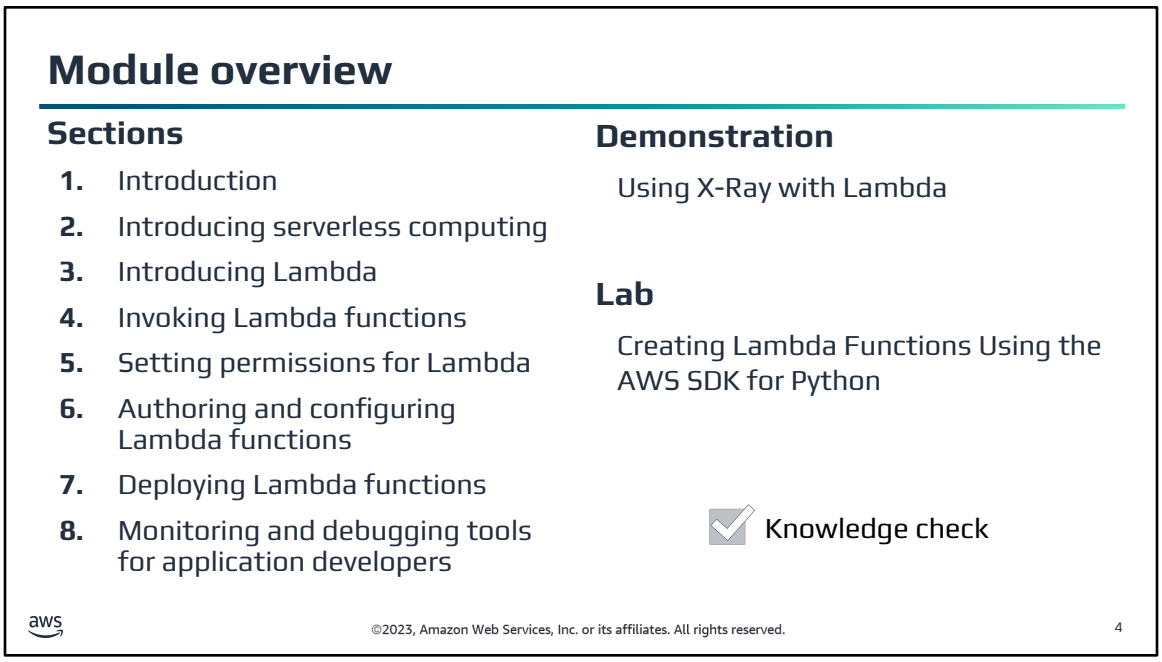
Welcome to Module 7: Developing Event-Driven Serverless Solutions.



Section 1: Introduction.



At the end of this module, you should be able to:
- Explain serverless computing
- Describe how Lambda works
- Recognize Lambda invocation models
- Identify how to use AWS Identity and Access Management (IAM) to grant Lambda permissions
- Indicate the steps to author and configure a Lambda function
- Explain how to deploy Lambda functions
- Identify why AWS X-Ray is a critical developer tool
- Create Lambda functions



This module includes the following sections:

1. Introduction
2. Introducing serverless computing
3. Introducing AWS Lambda
4. Invoking Lambda functions
5. Setting permissions for Lambda
6. Authoring and configuring Lambda functions
7. Deploying Lambda functions
8. Monitoring and debugging tools for application developers

This module also includes:

- A demonstration of using AWS X-Ray with a Lambda function.
- A lab where you create a Lambda function to query a database, call a deployment package, and configure a Lambda function. You then replace a mock endpoint with your Lambda function.

Finally, you will complete a knowledge check to test your understanding of key concepts covered in this module.

Café business requirement

Sofía finished using the mock endpoints to test the API, and now she wants to launch the dynamic version of the café website. She needs to update the API integration to connect to the backend database.



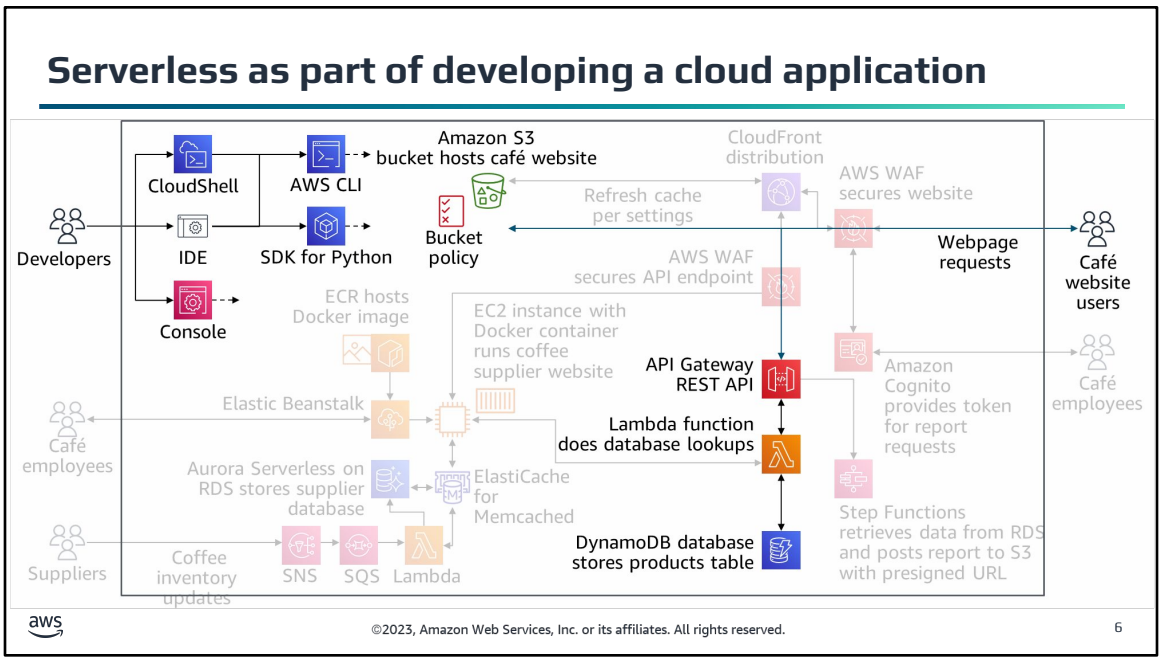


aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

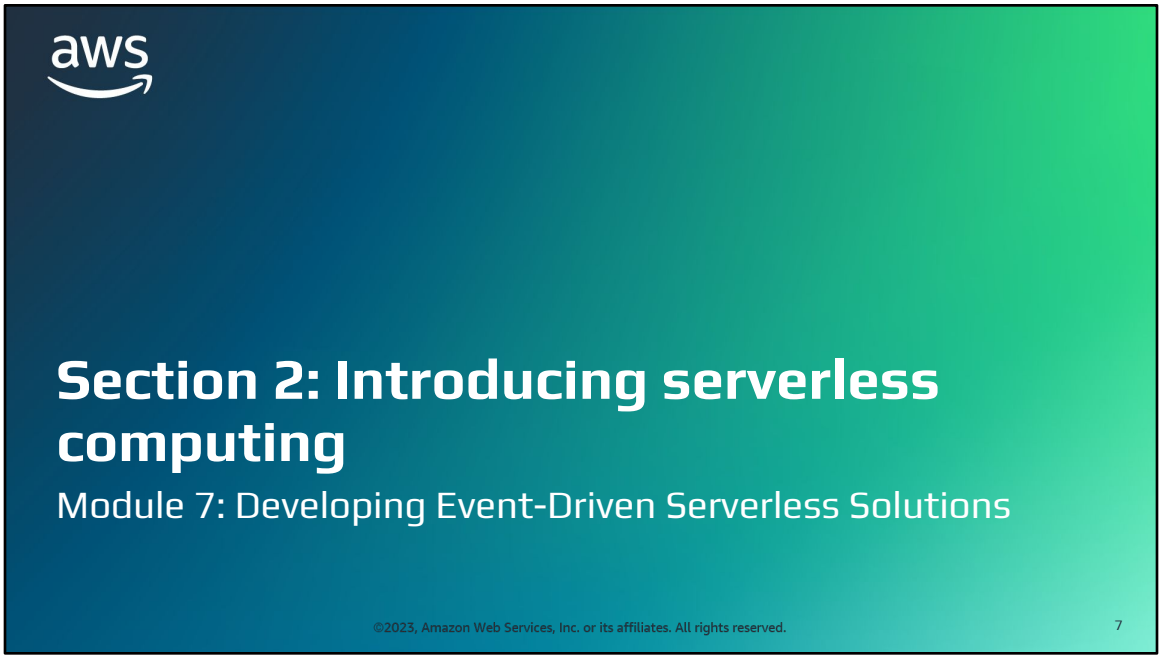
Sofía finished using the mock endpoints to test the API, and now she wants to launch the dynamic version of the café website. She needs to update the API integration to connect to the backend database.

Sofía will replace the mock endpoints with functional endpoints, and use a Lambda function to bridge the connection between the APIs and the data stored in DynamoDB.

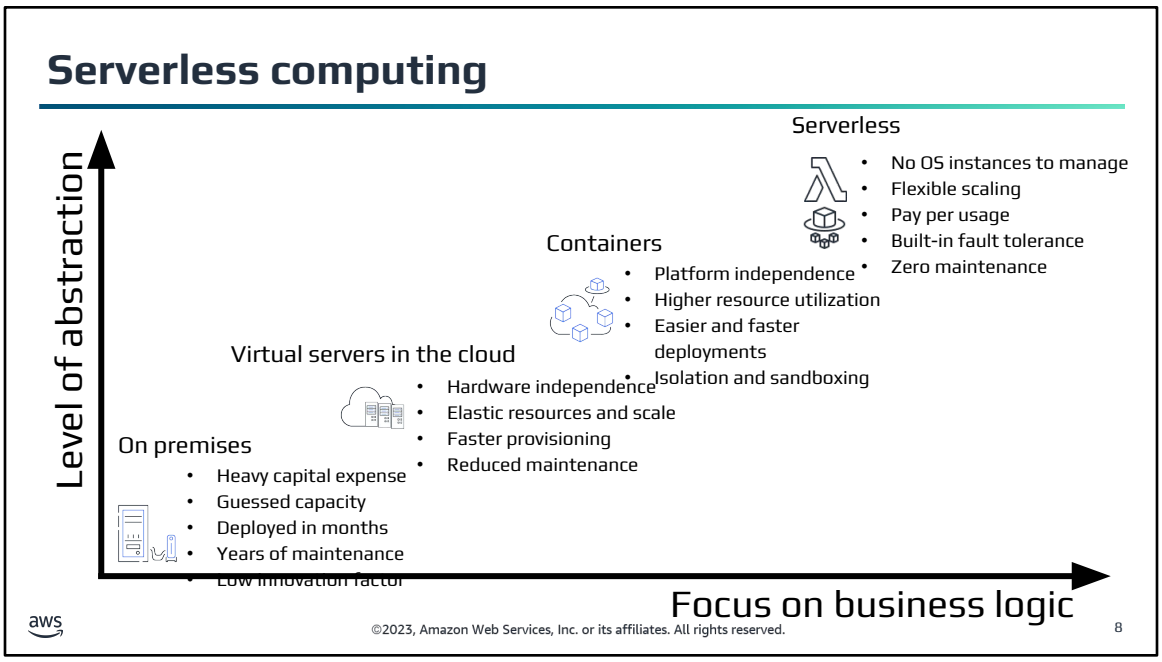


The diagram on this slide gives an overview of the application that you will build through the labs in this course. The highlighted portions are relevant to this module.

As highlighted in the diagram, you will configure calls to the API Gateway products endpoints to invoke a Lambda function that queries and returns data from DynamoDB.



Section 2: Introducing serverless computing.



One benefit of cloud computing is that you can abstract the infrastructure layer to alleviate many operational tasks. Thus, you can focus more on the business logic for your applications. You do not need to manage instances, operating systems, or servers. Everything that is required to run and scale your application with high availability is handled for you. Serverless computing eliminates infrastructure management tasks, such as server or cluster provisioning, patching, operating system maintenance, and capacity provisioning. The reduced overhead also means you can experiment and innovate faster.

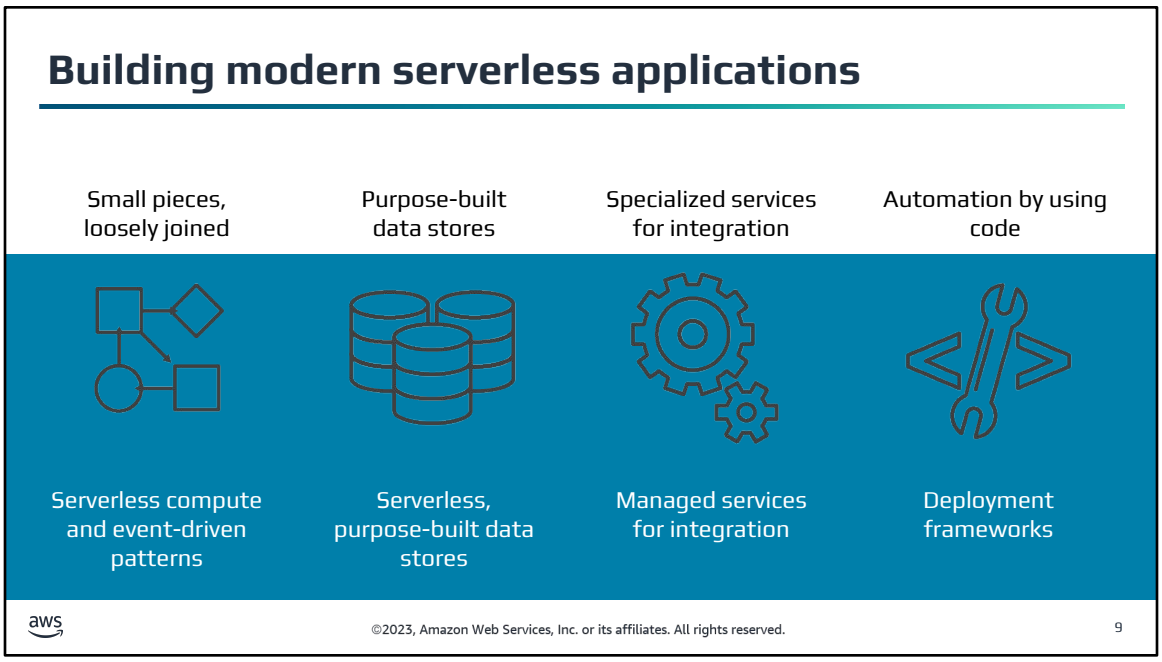
You don't pay for any infrastructure when the code isn't running. Additionally, with serverless computing, your code runs only when it's needed.

In a practical sense, with servers you must think about the following:

- Configuring instances
- Updating your OS
- Installing the application platform
- Configuring automatic scaling and load balancing
- Building and deploying applications
- Continuously patching, securing, and monitoring servers
- Monitoring and maintaining applications

With serverless, you can focus on the following:

- Building and deploying applications
- Monitoring and maintaining applications



From a builder's perspective, the abstraction of infrastructure into the cloud from virtual servers to containers and now to serverless has changed application development. It coincides with a move away from monolithic applications and a move toward service-oriented architectures and microservices.

The ideas of distributed applications and microservices aren't new, but the evolution of cloud services makes it easier to build applications that follow these principles:

- Use small, loosely joined pieces and purpose-built data stores. This strategy provides the flexibility to make changes without affecting other parts of the system, scale and deploy pieces independently, and incorporate reversible decisions.
- Use specialized integration services to increase the speed of delivery and decrease the effort of integrations between services.
- Write infrastructure as code to automate as much of the deployment process as you can. This approach makes it simpler to replicate environments, and deploy more frequently with minimal overhead and greater confidence.

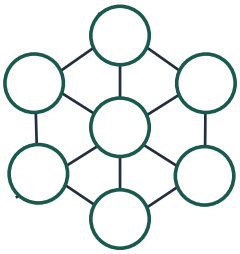
Serverless ("Serverless on AWS" at https://aws.amazon.com/serverless/) is an efficient approach to modern applications:

- With serverless compute services, you can focus on writing business logic while someone else handles scaling and high availability.
- With purpose-built serverless databases, you can choose the features and capacity that suit the type and volume of data that each service needs.
- Managed services provide API proxy, messaging, and orchestration and have prebuilt integrations with Lambda and other AWS services. With this process you can connect services by using asynchronous event-driven patterns to achieve decoupling and flexibility. At the same time, it reduces the undifferentiated lifting of building the integrations to connect your components.
- Serverless deployment frameworks make it easier to use infrastructure as code for deployments and automate the deployment pipeline.

Moving away from a monolithic application



Monolith
Does everything



Microservice
Does one thing

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

With a monolithic application, things start out easier. It's simpler to build and test, and you have only one artifact to deploy. For some less complex applications, a monolithic approach might be the best option for a long time.

For complex applications that grow over time, the interdependencies among components and the necessary coordination among development teams for testing and deploying become more difficult. These concerns can decrease developer productivity.

- You have one technology stack in which you must develop. You must hire and train developers in that technology and can't take advantage of skills in other languages. Any change to the technology stack becomes a major project.
- You have a single database. The most demanding use case drives your cost and capacity. Operations that aren't suited to the database or data structures will continually be tuned to strive for better performance. However, they will reach a limit that is impossible to improve on without a redesign.

From an operational standpoint, many developers are pushing changes through a shared release pipeline, which causes friction at many points of the lifecycle.

- If you want to upgrade a shared library to use a new feature, you must convince everyone else to upgrade at the same time.
- If you want to quickly push an important fix for your feature, you still must merge it with everyone else's in-process changes. This approach leads to merge Fridays or, worse yet, merge weeks when all developers compile their changes and resolve any conflicts for the next release.
- After development, you also face overhead when you're pushing the changes through the delivery pipeline. You must rebuild the entire application, run all of the test suites to ensure that you have no regression issues, and redeploy the entire application.

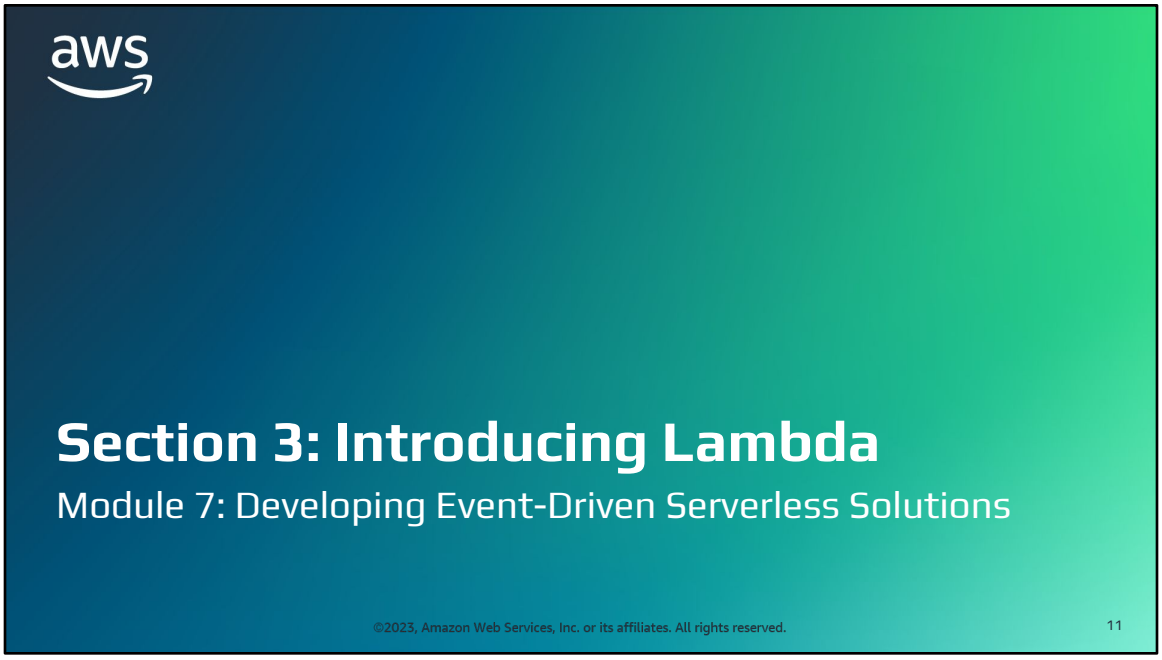
Every new feature adds to the complexity of the coordination, deployment, and ongoing operations effort. Technical debt stacks up, and it becomes nearly impossible to untangle code.

As an example, before migrating to a microservices approach, Amazon had a central team whose sole job was to deploy its monolithic application into production.

With a modern serverless application, the effort to get started is higher. Moving to microservice-based development requires a significant surge in effort as everyone learns and make mistakes. Often, everyone is new to the approaches that are required. Moving from experiments and utilities into full-scale production applications requires a surge of effort that includes both technology and organizational changes. The energy that it takes to get over that initial hurdle can hold organizations back without some initiating event or tipping point.

However, after the initial learning curve, the agility that this approach provides starts to accelerate the speed with which you can release new features. It also reduces the effort that it takes to keep them running.
Microservices provide these benefits:

- Scale individual services independently.
- Choose the best technology stack and data store for each service.
- Deploy changes independently as often as desired.
- Focus on a specific service and innovate within that service.



Section 3: Introducing Lambda.

AWS Lambda



AWS Lambda

Lambda is a serverless compute service that you can use to run code without provisioning or managing servers.

- Invokes your code in response to events
- Scales automatically
- Provides built-in code monitoring and logging with Amazon CloudWatch

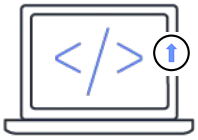
aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

AWS Lambda is the compute service for serverless.

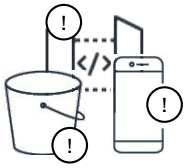
Lambda has the following advantages:
- Lets you run code without provisioning or managing servers
- Invokes your code in response to events that you configure
- Scales automatically based on demand
- Incorporates built-in monitoring and logging with Amazon CloudWatch

For more information about Lambda, see “AWS Lambda” at https://aws.amazon.com/lambda/.

How Lambda works



Upload your code to Lambda or write code in the Lambda editor



Set up your code to run when events occur in other AWS services, at HTTP endpoints, or as part of in-app activity



Lambda runs your code only when it is activated by an event and uses only the compute resources that are needed



Pay only for the compute time that you use

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Functions and event sources are the core components of Functions and event sources are the core components of Lambda. An *event source* is the entity that publishes events to Lambda. A Lambda *function* is the custom code that you provide, which processes the events. Lambda runs your function on your behalf.

To use Lambda, start by uploading existing code or writing code in the Lambda editor.

Then, set up your functions to run when events occur in other AWS service. Functions can also respond to incoming HTTP requests, consume events from a queue or stream, or run on a schedule.

Lambda runs your code only when activated and uses only the necessary compute resources. You pay only for the compute time that you use.

Lambda features



Offers the ability to bring your own code



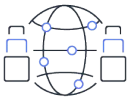
Integrates with and extends other AWS services



Offers flexible resource and concurrency models



Offers a flexible permissions model



Provides built-in availability and fault tolerance

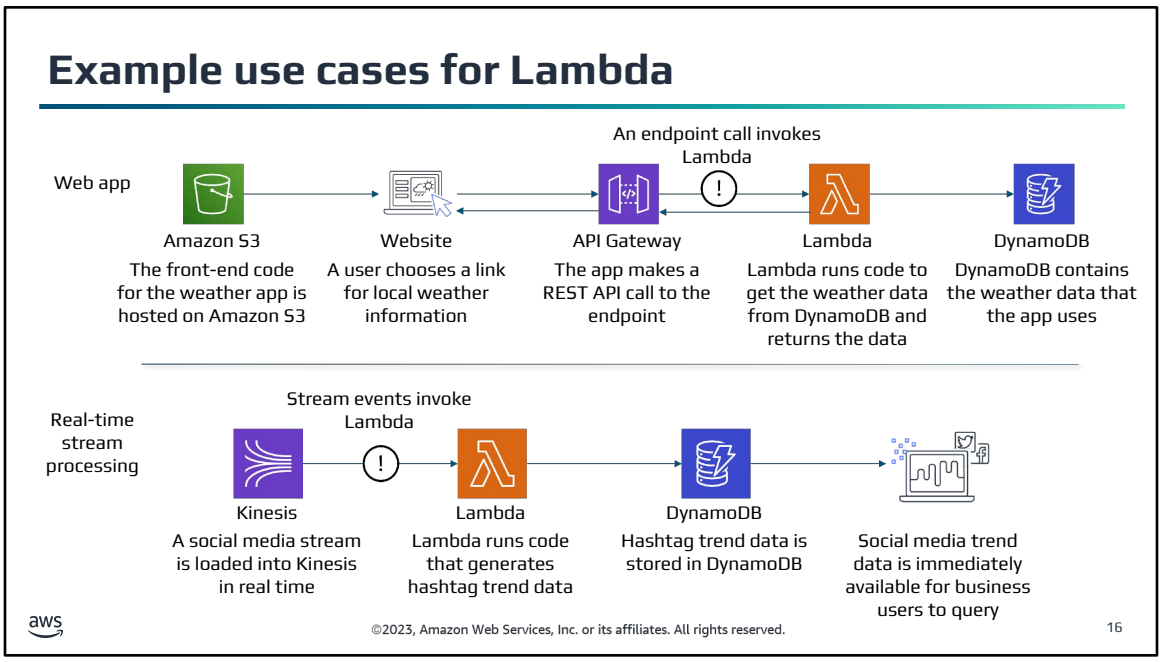


Reduces the need to pay for idle resources

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

AWS Lambda provides the following features:

- Offers the ability to bring your own code – The code that you write for Lambda isn't written in a new language that you have to learn. Development in Lambda is not tightly coupled to AWS, so you can easily port code in and out of AWS. With Lambda, you bring your code (written in your language of choice—such as Node.js, Java, Python, C#, Go, or Ruby), custom runtimes, and libraries.
- Integrates with and extends other AWS services – From within your Lambda function, you can do anything that traditional applications can do. Examples include calling an AWS SDK or invoking a third-party API.
- Offers flexible resource and concurrency models – Lambda scales in response to events. You configure memory settings, and AWS handles the details, such as CPU, network, and I/O throughput.
- Offers a flexible permissions model – Lambda uses AWS Identity and Access Management (IAM) to securely grant access to your resources and give you fine-grained control for invoking your functions.
- Provides built-in availability and fault tolerance – Because Lambda is a fully managed service, high availability and fault tolerance are integrated into the service, without any additional configuration on your part.
- Reduces the need to pay for idle resources – Lambda functions run only when they are invoked, so you don't pay for idle capacity.



You can use Lambda in various ways. Web applications and real-time stream processing are two common use cases. Additional common use cases include the following:

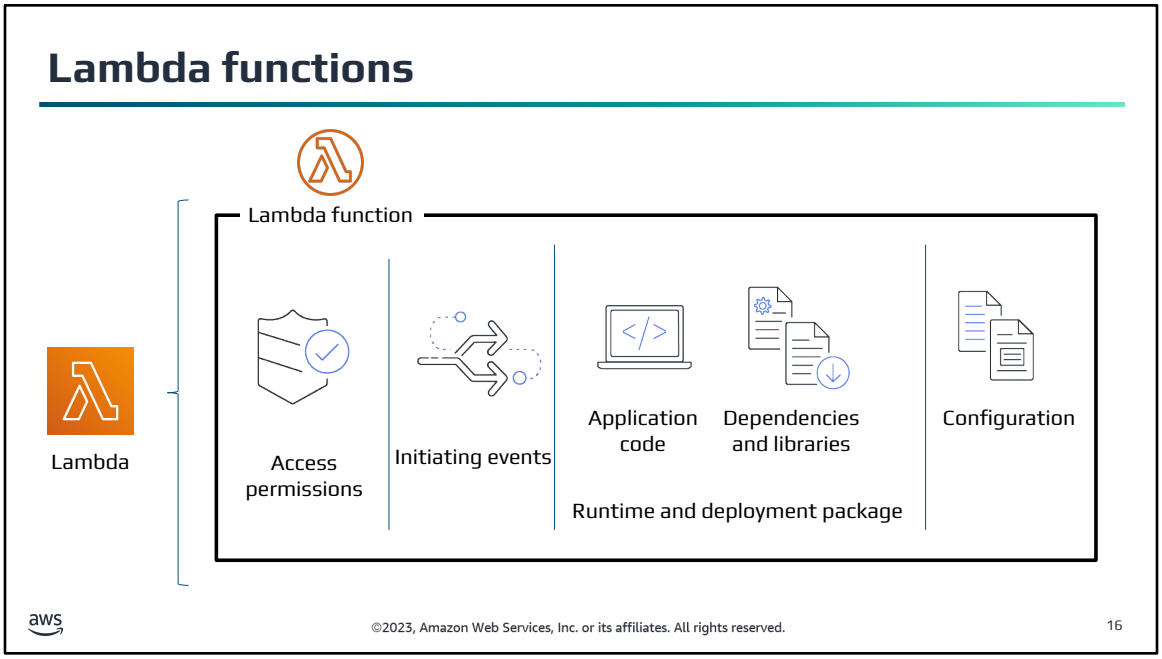
- **Backends** – You can build serverless backends by using Lambda to handle web, mobile, Internet of Things (IoT), and third-party API requests. You can build backends by using Lambda and Amazon API Gateway to authenticate and process API requests. Lambda makes it easy to create rich, personalized application experiences.
- **Data processing** – You can use Lambda to run code in response to triggers, such as changes in data, shifts in system state, or actions by users. AWS services, such as Amazon Simple Storage Service (Amazon S3) and Amazon DynamoDB, can directly invoke Lambda. Other services that can invoke Lambda include Amazon Kinesis, Amazon Simple Notification Service (Amazon SNS), and CloudWatch. AWS Step Functions can also orchestrate Lambda in workflows. You can use Lambda with these services to build various real-time processing systems for serverless data.
- **Chatbots** – Amazon Lex is a service for building conversational interfaces into any application by using voice and text. Amazon Lex uses the same deep learning technologies that power Amazon Alexa, and it integrates with Lambda.
- **Amazon Alexa** – You can use Lambda to build the backend for custom Alexa skills.
- **IT automation** – You can use Lambda to automate repetitive processes by invoking them with events or by running them on a fixed schedule. You can automatically update the firmware of hardware devices, start and stop Amazon Elastic Compute Cloud (Amazon EC2) instances, or schedule security group updates. In addition, you can automate your test and deployment pipeline.

For more case studies, see “AWS Lambda” at https://aws.amazon.com/lambda/.

Anything that you want to do with servers you can probably do with Lambda. Use Lambda and serverless to reimagine your approach to new or updated applications.

To get started, follow these suggestions:

- Look for opportunities to introduce serverless through common use cases.
- Choose something authentic, iterate on it, learn from it, and expect to keep working on it in production.
- Expect to change your mind about what you thought was the best solution when you started. Keep learning and updating your approach.



As a developer, you create Lambda functions that the Lambda service manages. When you create a function, you define the permissions for the function and specify which events will invoke the function. You choose a runtime and create a deployment package that includes your application code and any dependencies and libraries necessary to run your code. Finally, you configure parameters such as memory, timeout, and concurrency. When your function is invoked, Lambda provides a runtime environment based on the runtime and configuration options that you selected.

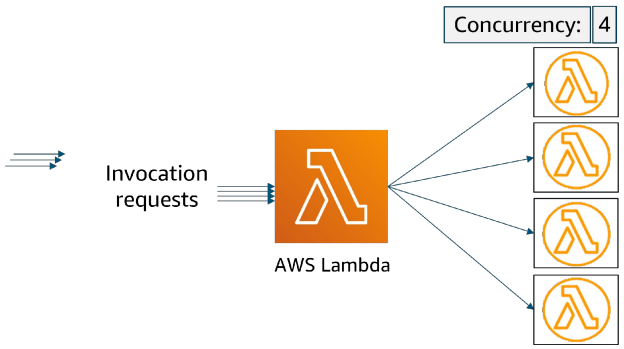
Lambda initializes concurrent environments to maintain the pace of invocation requests up to certain quotas. Each environment is temporary, and the Lambda service controls it, which affects some of the best practices that you should apply to your function code. You will discuss these ideas later in this section.

Lambda supports multiple languages through the use of runtimes, including Node.js, Python, Ruby, Java, Go, and .NET. You can also implement a custom runtime if you want to use another language in Lambda.

For more information about the Lambda runtime environment, see “Lambda Runtimes” at https://docs.aws.amazon.com/lambda/latest/dg/current-supported-versions.html.

Lambda functions run in on-demand ephemeral environments

Concurrency: The number of function invocations that are running at one time



aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Each invocation of a Lambda function runs in an ephemeral environment—one that is initialized on demand, lasts a brief time, and then is removed. Lambda manages creating and tearing down these environments for your functions. You don't have direct control over these environments.

Concurrency is the number of Lambda function invocations that are running at one time. Lambda provides the environments on demand as it receives requests, which is how Lambda quickly scales horizontally.

Factors that influence concurrency include the following:

- **Reserved concurrency** on a function – This optional value is set per function and reserves a subset of the Regional quota for the function. It also establishes the maximum concurrent instances that are permitted for the function. For more information about managing reserved concurrency, see "Configuring reserved concurrency" at https://docs.aws.amazon.com/lambda/latest/dg/configuration-concurrency.html.
- **Regional quota** – This quota is the total number of invocations that can run concurrently across all Lambda functions within an account by Region. AWS sets this number as a soft quota on the account. For more information about quotas, see "Lambda Quotas" at https://docs.aws.amazon.com/lambda/latest/dg/gettingstarted-limits.html.
- **Burst quota** – Each Region has a burst limit, which prevents concurrency from increasing too quickly if a large spike of requests occurs in a short time period. You cannot modify this limit. For more information about bursts, see "Lambda function scaling" at https://docs.aws.amazon.com/lambda/latest/dg/invocation-scaling.html.
- **Request rate and function duration** – This value is the rate at which new invocation requests for the function arrive, in conjunction with how long Lambda takes to run the function.
- **The event source** – Lambda manages concurrency of requests in different ways based on the type of event that invokes it.

Section 3 key takeaways



- Lambda is a serverless compute service that integrates with other AWS services.
- You can use Lambda for many use cases including web applications, backends, and data processing.
- Lambda functions run in on-demand temporary environments.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Some key takeaways from this section of the module include the following:

- Lambda is a serverless compute service that integrates with other AWS services.
- You can use Lambda for many use cases including web applications, backends, and data processing.
- Lambda functions run in on-demand temporary environments.